Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №3

Технологія розроблення програмного забезпечення

«Діаграма розгортання. Діаграма компонентів. Діаграма взаємодій та послідовностей.»

Варіант 11

Виконала
студентка групи ІА-24
Тильна Марія Сергіївна

Перевірив:
Мягкий Михайло Юрійович

Київ 2024р.

Зміст

1. Теоретичні відомості_____2
2. Завдання_____3
3. Діаграма розгортання_____4
4. Діаграма компонентів_____5
5. Діаграма послідовностей_____8
6. Висновок_____10

Мета: навчитися створювати діаграми розгортання, компонентів та послідовностей у нотації UML для моделювання програмних систем.

Теоретичні відомості

Діаграма розгортання

Діаграма розгортання показує фізичне розташування програмних компонентів на апаратному забезпеченні системи. Вона відображає вузли (пристрої або сервери), де запускаються програмні модулі, та зв'язки між ними. Основною метою діаграми розгортання є візуалізація того, як система буде працювати в реальному середовищі. Вона демонструє, на яких пристроях або серверах розміщуються різні частини системи (клієнтська частина, сервер додатків, база даних) та як вони взаємодіють між собою. Це корисно для розуміння фізичної архітектури системи, її масштабованості та ефективності.

Діаграма компонентів

Діаграма компонентів використовується для відображення логічної структури програмного забезпечення та показує, як різні модулі або компоненти системи взаємодіють між собою. Компоненти — це окремі частини програмного забезпечення, які виконують конкретні функції та можуть взаємодіяти через інтерфейси. Ця діаграма допомагає зрозуміти структуру системи, які модулі вона містить, та як ці модулі обмінюються інформацією. Використовується для документування системи на етапі проектування та дає уявлення про логічну організацію коду.

Діаграма послідовностей

Діаграма послідовностей показує, як об'єкти системи взаємодіють між собою в хронологічному порядку для виконання певної операції. Вона фокусується на тому, які повідомлення передаються між компонентами та в якому порядку. Це корисний інструмент для моделювання динамічних аспектів системи, таких як процес реєстрації користувача або збереження даних. Діаграма допомагає зрозуміти, як відбувається взаємодія між клієнтом, сервером та базою даних у реальному часі, забезпечуючи чітке уявлення про порядок виконання операцій.

Завдання

1. Ознайомитися з короткими теоретичними відомостями.
2. Розробити діаграму розгортання для проектованої системи.
3. Розробити діаграму компонентів для проектованої системи.
4. Розробити діаграму послідовностей для проектованої системи.
5. Скласти звіт про виконану роботу

..11 Web crawler (proxy, chain of responsibility, memento, template method, composite, p2p)

Веб-сканер повинен вміти розпізнавати структуру сторінок сайту, переходити за посиланнями, збирати необхідну інформацію про зазначений термін, видаляти не семантичні одиниці (рекламу, об'єкти javascript і т.д.), зберігати знайдені дані у вигляді структурованого набору html файлів вести статистику відвіданих сайтів і метадані.

.

Хід роботи

1. Діаграма розгортання

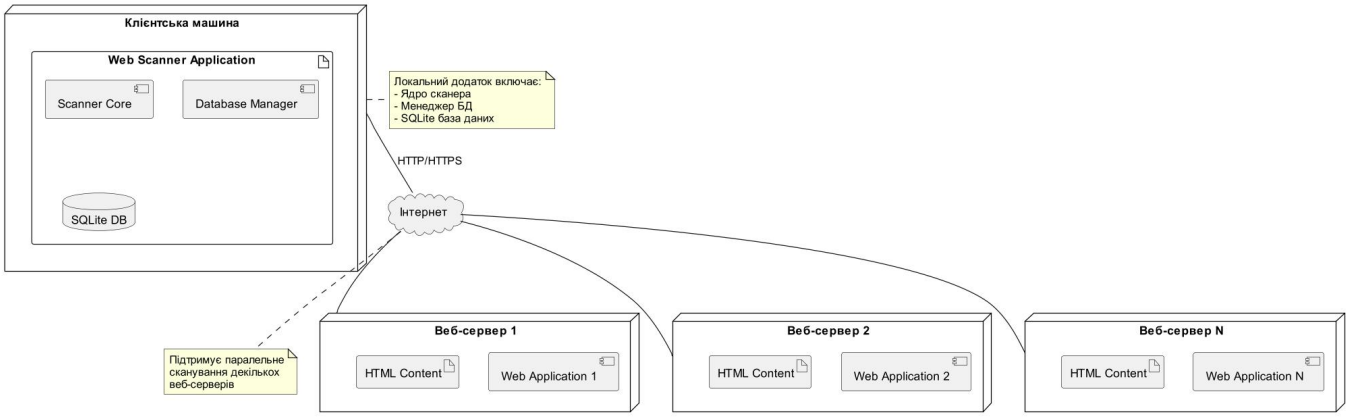

Рис.1 Діаграма розгортання

Ця діаграма відображає структуру розгортання додатку WebCrawler. Вона демонструє основні компоненти системи, їх зв'язки, та взаємодію між різними вузлами розгортання.

Компоненти діаграми
Клієнтська машина:
Локальна машина, на якій працює додаток Web Scanner.
Містить такі основні компоненти:

Ядро сканера (Scanner Core) — головний функціональний модуль, який виконує логіку сканування, аналізу і збору даних.
Менеджер бази даних (Database Manager) — модуль, відповідальний за управління збереженням даних, включаючи записи, оновлення та видалення.
SQLite база даних (SQLite DB) — локальна база даних для збереження результатів сканування, статистики та метаданих.

Веб-сервери:

Веб-сервер 1 (Web Application 1), Веб-сервер 2 (Web Application 2), Веб-сервер N (Web Application N):
Ці сервери представляють цільові веб-сайти, які будуть скануватися.
Кожен сервер містить:

- **Веб-додаток (Web Application)** — програмний модуль, що обробляє запити.
- **HTML-вміст (HTML Content)** — веб-сторінки, які скануються.

Інтернет:

- Середовище, через яке здійснюється передача даних між клієнтською машиною та веб-серверами.
- Підтримує паралельне сканування декількох серверів.

Зв’язки між вузлами

- Клієнтська машина взаємодіє з Інтернетом за допомогою протоколів HTTP/HTTPS для надсилання запитів і отримання відповідей.
- Інтернет виступає посередником, забезпечуючи доступ до цільових веб-серверів.
- Веб-сервери обробляють запити клієнтської машини, повертаючи HTML-вміст для подальшої обробки.

Особливості системи

Модульність: Архітектура додатку розділена на чіткі компоненти (ядро сканера, менеджер БД), що спрощує обслуговування та розширення.
Локальне зберігання: Використання SQLite забезпечує автономність і ефективне управління даними.
Масштабованість: Система може одночасно обробляти дані з декількох серверів.

2. Діаграма компонетів

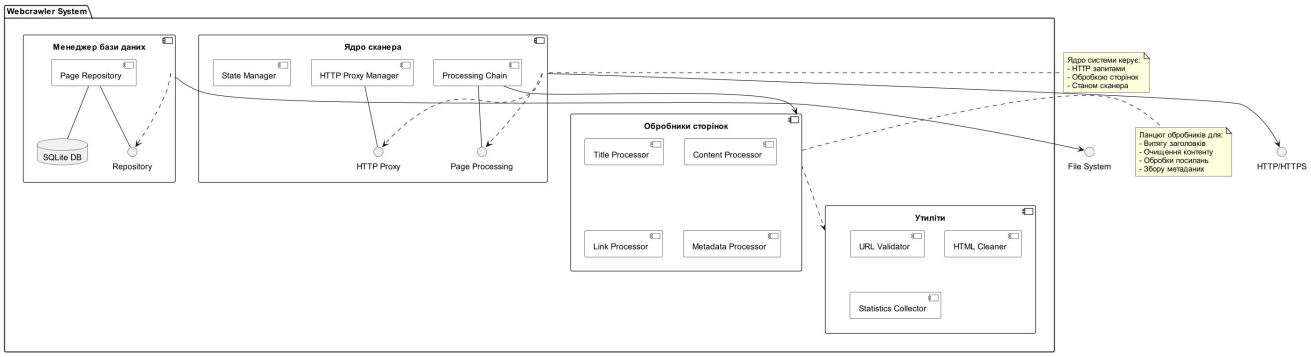

Рис.2 Діаграма компонентів

Діаграма компонентів демонструє основні модулі системи, їхні обов'язки, залежності та зв'язки. Це дозволяє зрозуміти, як різні частини системи взаємодіють одна з одною.

Структура

Усі компоненти об'єднані в пакет Webcrawler System, який включає модулі, що забезпечують роботу сканера.

Основні компоненти

Ядро сканера (Scanner Core):

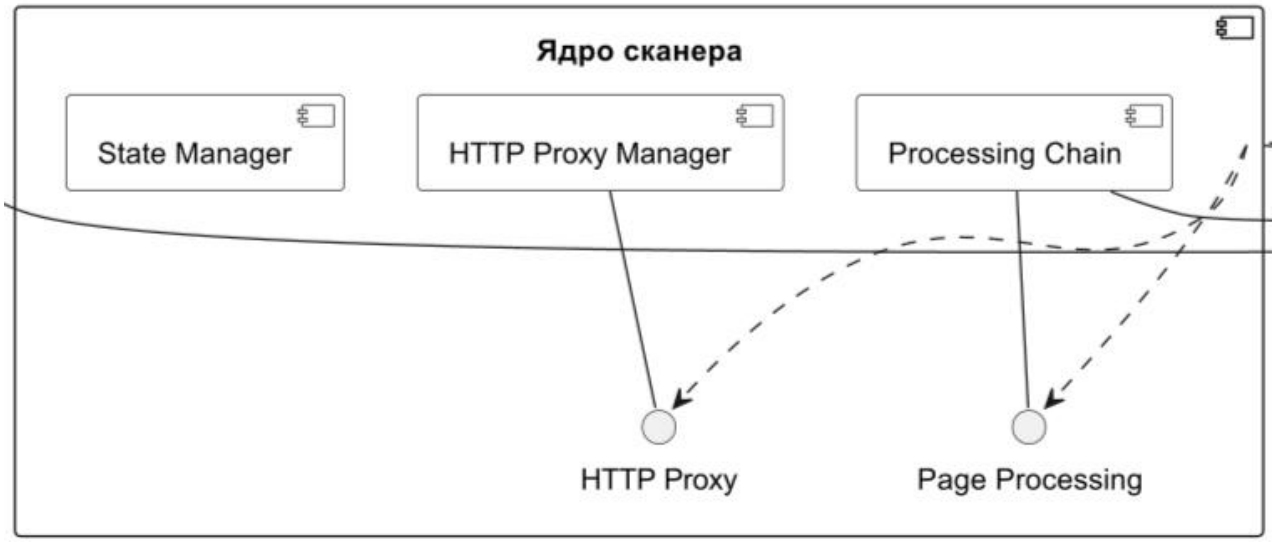

Рис2 Діарама компонентів

Виконує ключові функції системи:

- Управління HTTP-запитами через HTTP Proxy Manager.
- Обробка сторінок за допомогою Processing Chain.
- Відстеження стану сканера через State Manager.

Інтерфейси:

- HTTP Proxy — для управління підключенням до веб-серверів.
- Page Processing — для обробки вмісту сторінок.

Менеджер бази даних (Database Manager):

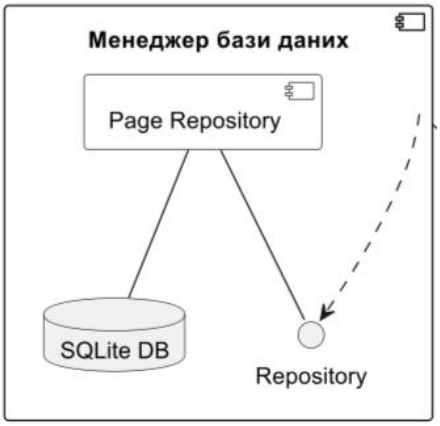

Рис2.1 Менеджер бази даних

Відповідає за зберігання та доступ до даних.
Page Repository працює як проміжний шар між додатком і локальною базою даних SQLite DB.

Інтерфейси:
Repository — для запитів до сховища сторінок.

Обробники сторінок (Page Processors):

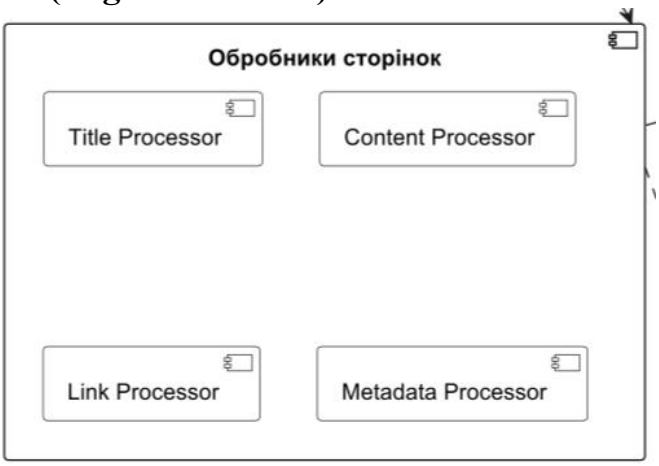

Рис2.2 Обробники сторінок

Модуль, який реалізує логіку обробки HTML-сторінок:

- Title Processor — витягує заголовки.
- Content Processor — очищує і обробляє вміст сторінок.
- Link Processor — аналізує та зберігає посилання.
- Metadata Processor — витягує додаткові дані (наприклад, ключові слова або час створення сторінки).

Утиліти (Utilities):

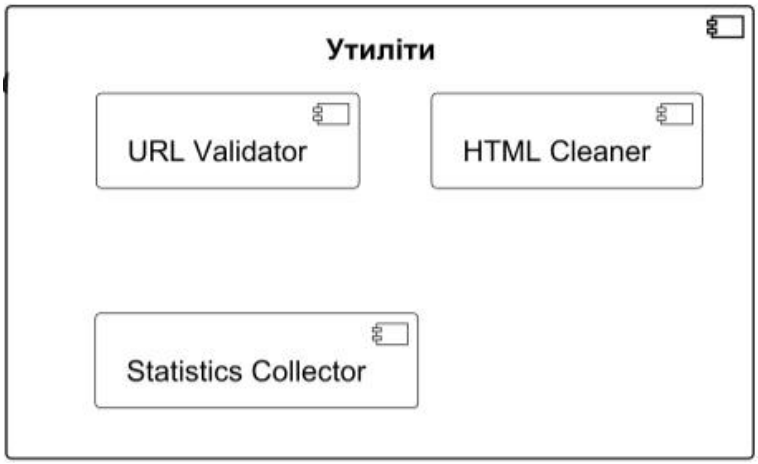

Рис2.3 Утиліти

Допоміжні компоненти для підтримки основних функцій:

- URL Validator — перевіряє валідність URL.
- HTML Cleaner — очищує HTML-код від непотрібних тегів і символів.
- Statistics Collector — збирає статистику про виконання сканера.

3. Діаграма послідовностей

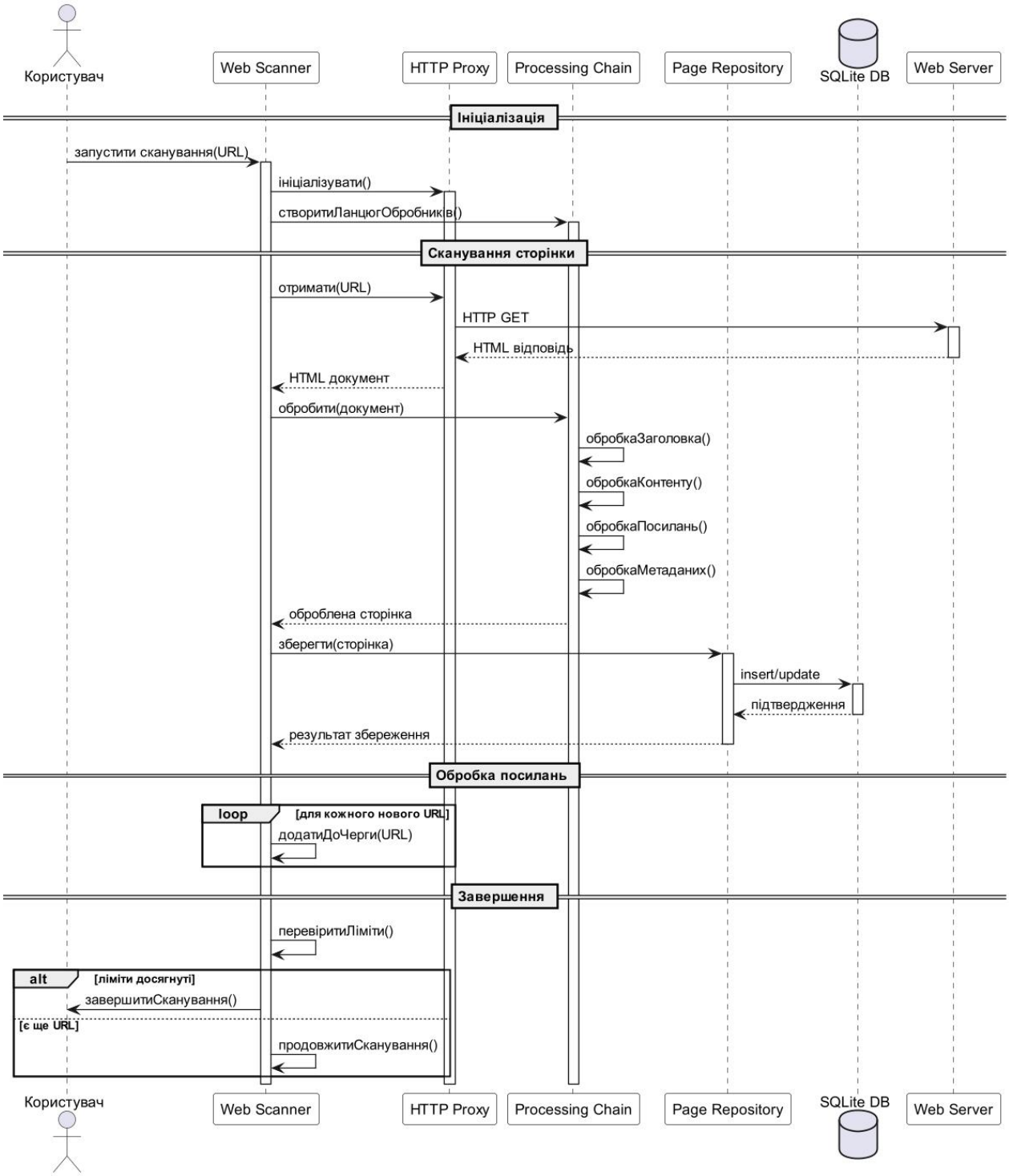

Рис.3 Діаграма послідовностей

Діаграма послідовностей відображає динаміку роботи системи, демонструючи взаємодію між користувачем, основними компонентами сканера та зовнішніми системами (веб-серверами і базою даних). Вона ілюструє, як інформація передається і обробляється в рамках одного циклу сканування.

Основні етапи процесу

Ініціалізація:

1. **Користувач** запускає сканування, передаючи URL початкової сторінки.
2. **Web Scanner** виконує ініціалізацію ключових компонентів:
    1. **HTTP Proxy** — для взаємодії з веб-серверами.
    2. **Processing Chain** — для обробки HTML-сторінок через ланцюг процесорів.

Сканування сторінки:

1. **Web Scanner** звертається до **HTTP Proxy**, щоб надіслати запит на отримання HTML-контенту.
2. **HTTP Proxy** взаємодіє із зовнішнім **веб-сервером**, отримуючи HTML-відповідь, яку повертає **Web Scanner**.
3. **Processing Chain** обробляє отриманий документ через кілька етапів:
    1. Аналіз заголовків.
    2. Очищення та обробка контенту.
    3. Витяг посилань для подальшого сканування.
    4. Збір метаданих.
4. Після обробки сторінка повертається **Web Scanner**.

Збереження результатів:

1. **Web Scanner** зберігає оброблену сторінку через **Page Repository** у локальну базу даних **SQLite DB**.
2. **Page Repository** передає дані до бази, отримує підтвердження і повертає результат збереження.

Обробка посилань:

1. Всі знайдені нові URL додаються до черги сканування для подальшого оброблення.

Завершення:

1. **Web Scanner** перевіряє, чи досягнуто лімітів сканування (наприклад, кількість сторінок, час або глибина).
2. **Умови**:
    1. Якщо ліміти досягнуті, система повідомляє користувача про завершення.
    2. Якщо черга містить нові URL, сканування триває.

Особливості діаграми

- **Ітеративність**:
    - Чітко показаний процес циклічної обробки нових URL до досягнення заданих лімітів.
- **Модульність**:
    - Логіка роботи розділена між компонентами (наприклад, Proxy, Chain, Repo), кожен з яких виконує свою роль.
- **Інтеграція**:
    - Взаємодія з зовнішніми системами, такими як веб-сервери і база даних, чітко відображена.

Висновок: У ході виконання цієї лабораторної роботи я ознайомилась з UML-діаграм, зокрема ознайомившись з діаграмами розгортання, компонентів та послідовностей.